



```
In [12]: import pandas as pd
import seaborn as sns
```

```
In [13]: import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")
```

```
In [14]: data = pd.read_csv('Social_Network_Ads.csv')
```

```
In [15]: data
```

```
Out[15]:
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

400 rows × 5 columns

```
In [16]: data.shape
```

```
Out[16]: (400, 5)
```

```
In [17]: data.isnull().sum()
```

```
Out[17]: User ID      0
Gender      0
Age         0
EstimatedSalary  0
Purchased   0
dtype: int64
```

```
In [18]: data.dtypes
```

```
Out[18]: User ID      int64
Gender      object
Age         int64
EstimatedSalary  int64
Purchased   int64
dtype: object
```

```
In [19]: data.select_dtypes(include=['object']).nunique()
```

```
Out[19]: Gender      2
dtype: int64
```

```
In [20]: x = data.iloc[:,2:4].values
y = data.iloc[:,4:5].values
```

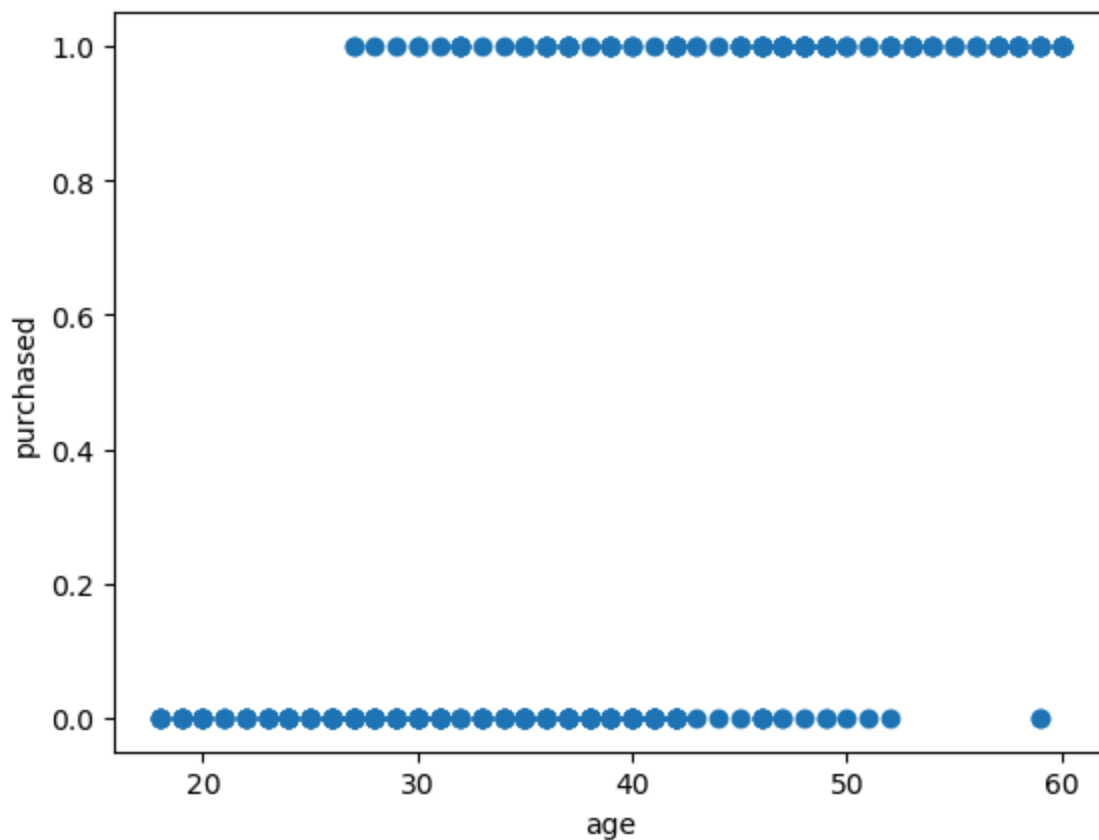
```
In [21]: data.describe()
```

```
Out[21]:
```

	User ID	Age	EstimatedSalary	Purchased
count	4.000000e+02	400.000000	400.000000	400.000000
mean	1.569154e+07	37.655000	69742.500000	0.357500
std	7.165832e+04	10.482877	34096.960282	0.479864
min	1.556669e+07	18.000000	15000.000000	0.000000
25%	1.562676e+07	29.750000	43000.000000	0.000000
50%	1.569434e+07	37.000000	70000.000000	0.000000
75%	1.575036e+07	46.000000	88000.000000	1.000000
max	1.581524e+07	60.000000	150000.000000	1.000000

```
In [22]: from math import exp
from sklearn.model_selection import train_test_split
```

```
In [23]: plt.scatter(data['Age'], data['Purchased'])
plt.xlabel("age")
plt.ylabel("purchased")
plt.show()
```



```
In [24]: X_train, X_test, y_train, y_test = train_test_split(data['Age'], data['Purchased'],
```

```
In [25]: from sklearn.linear_model import LogisticRegression
```

```
In [26]: model = LogisticRegression()  
model.fit(X_train.values.reshape(-1, 1), y_train)
```

```
Out[26]: 

▼ LogisticRegression ⓘ ?



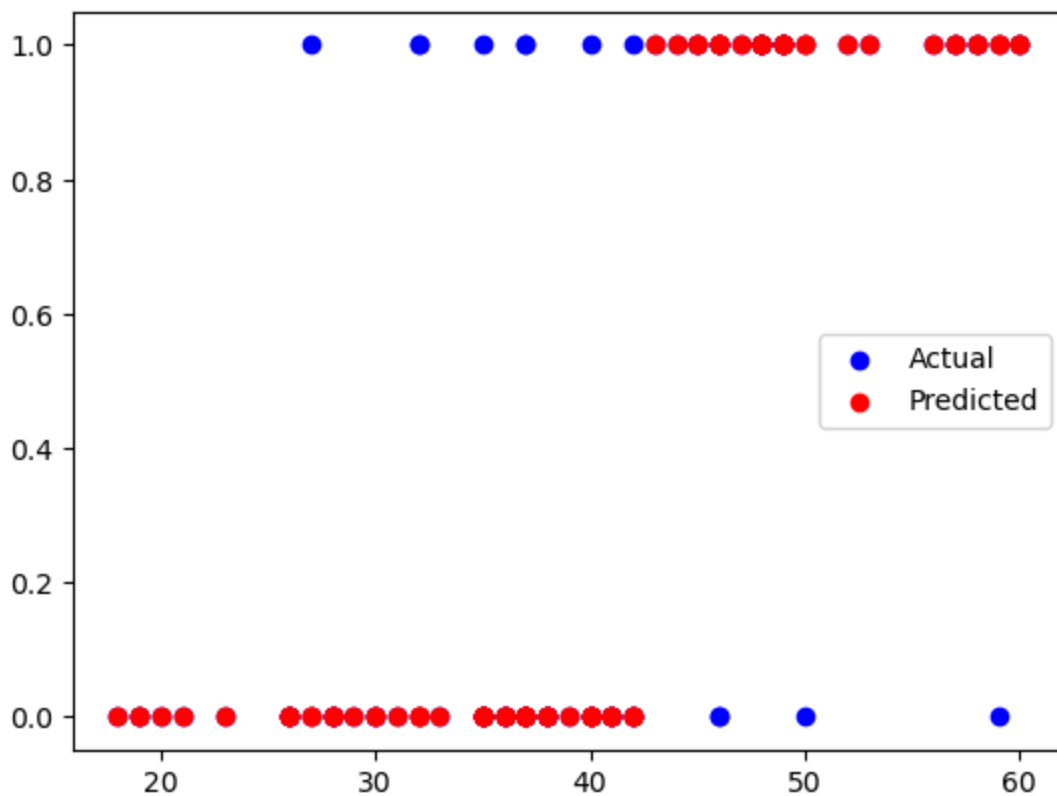
LogisticRegression()


```

```
In [27]: y_pred_sk = model.predict(X_test.to_frame())
```

```
In [28]: y_pred_sk = model.predict(X_test.values.reshape(-1, 1))
```

```
In [29]: plt.scatter(X_test, y_test, color='blue', label='Actual')  
plt.scatter(X_test, y_pred_sk, color='red', label='Predicted')  
plt.legend()  
plt.show()
```



```
In [30]: from sklearn.metrics import confusion_matrix
```

```
In [31]: from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred_sk)
tn, fp, fn, tp = cm.ravel()
print("True Negatives:", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives:", tp)
```

```
True Negatives: 42
False Positives: 4
False Negatives: 8
True Positives: 26
```

```
In [32]: accuracy = (tn + tp) / (tn + fp + fn + tp)
print("Accuracy: {:.2f}%".format(accuracy * 100))
```

```
Accuracy: 85.00%
```

```
In [33]: precision = tp / (tp + fp)
print("Precision: {:.2f}".format(precision))
```

```
Precision: 0.87
```

```
In [34]: #Error rate
err = (fp + fn) / (tp + tn + fn + fp)
print("Error rate {:.2f}".format(err))
```

```
Error rate 0.15
```